

MINI PROYECTO EVALUABLE

Modulo 2: Normalizacion de un Sistema de Ventas

Bases de Datos Relacionales con PostgreSQL

Guia para que el aprendiz transforme una tabla de ventas desnormalizada en un modelo relacional consistente, aplicando 1FN, 2FN y 3FN.

Campo	Detalle
Duracion sugerida	4 a 6 horas de trabajo practico
Modalidad	Individual o parejas
Herramientas	PostgreSQL, pgAdmin o DBeaver, Docker opcional, diagrams.net / Draw.io
Producto final	Informe de normalizacion, diagrama ER, scripts SQL y consultas de validacion
Nivel	Modulo 2 - Normalizacion

1. Contexto del mini proyecto

La empresa ficticia Ventas Express S.A.S. registra sus ventas en una hoja de calculo. El archivo funciona para capturar datos, pero presenta redundancia, errores de actualizacion, dificultades para consultar informacion y riesgo de inconsistencias. El equipo de tecnologia requiere convertir esa estructura en una base de datos relacional normalizada para PostgreSQL.

Problema central:

- Una misma venta puede incluir varios productos, pero en la tabla original los productos estan agrupados en campos separados por comas.
- Los datos de clientes, vendedores, categorias, productos y medios de pago se repiten en cada registro.
- El total de la venta se guarda como dato capturado, aunque puede calcularse desde el detalle.
- No existen claves primarias, claves foraneas ni restricciones de integridad.
- El modelo actual dificulta responder consultas basicas como ventas por vendedor, productos mas vendidos o historial de compras por cliente.

2. Objetivos de aprendizaje

- Identificar problemas de redundancia, dependencia parcial, dependencia transitiva y atributos no atomicos.
- Aplicar Primera Forma Normal (1FN), Segunda Forma Normal (2FN) y Tercera Forma Normal (3FN).
- Definir entidades, atributos, claves primarias, claves foraneas y relaciones cardinales.
- Diseñar un modelo entidad-relacion para un sistema de ventas.
- Implementar el modelo fisico en PostgreSQL usando DDL, restricciones y datos de prueba.
- Validar que el modelo normalizado permite reconstruir la informacion original sin perdida de datos.

3. Reglas de negocio del sistema de ventas

1. Un cliente puede realizar muchas ventas; cada venta pertenece a un unico cliente.
2. Un vendedor puede atender muchas ventas; cada venta es atendida por un unico vendedor.
3. Una venta puede contener uno o varios productos.
4. Un producto puede aparecer en muchas ventas.
5. Cada producto pertenece a una categoria.
6. El precio unitario usado en una venta debe conservarse en el detalle, porque el precio del producto puede cambiar con el tiempo.
7. El descuento se registra por linea de detalle, no por producto general.
8. El medio de pago se registra para cada venta.
9. El total de la venta debe poder calcularse desde el detalle: $\text{cantidad} * \text{precio_unitario} - \text{descuento}$.
10. No debe permitirse registrar un detalle de venta con producto inexistente, venta inexistente o cantidad menor o igual a cero.

4. Tabla inicial desnormalizada

La siguiente tabla representa el punto de partida. Debe cargarse como tabla cruda para analizar sus problemas. No se debe usar como modelo final.

Columna	Descripcion del problema
productos_codigos	Contiene varios codigos en una sola celda; incumple atomicidad.
productos_nombres	Repite nombres y depende del codigo de producto, no de la venta.
categorias	Repite categorias y depende del producto.
cantidades	Contiene varias cantidades asociadas por posicion a productos_codigos.
precios_unitarios	Contiene varios precios asociados por posicion a productos_codigos.
cliente_nombre, cliente_email, cliente_telefono	Dependen del cliente, no de la venta.
vendedor_nombre, vendedor_zona	Dependen del vendedor, no de la venta.
total_venta	Es un dato calculable; puede generar inconsistencias.

5. Script base para crear la tabla cruda

Crea una base de datos para el ejercicio y ejecuta el siguiente script de carga inicial:

```
-- Base sugerida: ventas_normalizacion
-- Este script crea una tabla cruda con errores intencionales de disenio.

DROP TABLE IF EXISTS ventas_crudas;

CREATE TABLE ventas_crudas (
    venta_id          VARCHAR(10),
    fecha_venta       DATE,
    cliente_doc        VARCHAR(20),
    cliente_nombre     VARCHAR(100),
    cliente_email      VARCHAR(120),
    cliente_telefono   VARCHAR(30),
    cliente_direccion  VARCHAR(150),
    cliente_ciudad     VARCHAR(80),
    vendedor_id        VARCHAR(10),
    vendedor_nombre    VARCHAR(100),
    vendedor_zona      VARCHAR(80),
    productos_codigos  TEXT,
    productos_nombres  TEXT,
    categorias         TEXT,
    cantidades         TEXT,
    precios_unitarios  TEXT,
    descuentos         TEXT,
    metodo_pago        VARCHAR(80),
    entidad_pago       VARCHAR(80),
    total_venta        NUMERIC(12,2)
);

INSERT INTO ventas_crudas VALUES
('V1001', '2026-04-01', 'CC101', 'Maria Gomez', 'maria.gomez@mail.com', '3101112233',
'Calle 10 # 5-20', 'Bogota', 'VEN01', 'Ana Torres', 'Norte',
'P001,P002', 'Mouse USB,Teclado mecanico', 'Perifericos,Perifericos',
'2,1', '45000,180000', '0,0', 'Transferencia', 'Bancolombia', 270000),

('V1002', '2026-04-02', 'CC102', 'Juan Perez', 'juan.perez@mail.com', '3155558899',
```

```
'Carrera 8 # 20-15', 'Bogota', 'VEN02', 'Carlos Ruiz', 'Centro',
'P003,P001', 'Monitor 24,Mouse USB', 'Pantallas,Perifericos',
'1,1', '720000,45000', '20000,0', 'Tarjeta credito', 'Visa', 745000),

('V1003', '2026-04-03', 'CC101', 'Maria Gomez', 'maria.gomez@mail.com', '3101112233',
'Calle 10 # 5-20', 'Bogota', 'VEN01', 'Ana Torres', 'Norte',
'P004,P002,P005', 'Portatil 14,Teclado mecanico,Base refrigerante',
'Computadores,Perifericos,Accesorios', '1,2,1', '2450000,180000,95000',
'50000,0,0', 'Transferencia', 'Bancolombia', 2855000),

('V1004', '2026-04-04', 'CC103', 'Laura Rojas', 'laura.rojas@mail.com', '3209994455',
'Av. 68 # 45-30', 'Medellin', 'VEN03', 'Diana Mora', 'Occidente',
'P003,P006', 'Monitor 24,Webcam HD', 'Pantallas,Perifericos',
'2,1', '720000,150000', '0,10000', 'Efectivo', 'Caja principal', 1580000);
```

6. Actividades paso a paso

Paso 1. Diagnostico de la tabla cruda

- Identifica atributos no atomicos.
- Marca datos repetidos por cliente, vendedor, producto, categoria y pago.
- Explica al menos tres anomalias: insercion, actualizacion y eliminacion.
- Redacta una lista inicial de dependencias funcionales.

Paso 2. Aplicacion de 1FN

- Transforma los campos multivaluados en registros separados.
- Cada fila debe representar una venta con un solo producto asociado.
- Verifica que no existan listas separadas por comas ni grupos repetitivos.
- Propone una clave compuesta provisional para la tabla en 1FN.

Paso 3. Aplicacion de 2FN

- Evalua si la tabla en 1FN usa una clave compuesta.
- Separa los atributos que dependen solo de una parte de la clave.
- Identifica entidades como venta, cliente, vendedor, producto y detalle de venta.
- Define claves primarias y foraneas para las entidades resultantes.

Paso 4. Aplicacion de 3FN

- Elimina dependencias transitivas.
- Separa categorias de productos si la categoria depende del producto y no de la venta.
- Separa ciudades o zonas solo si justificas que se administraran como catalogos.
- Revisa que cada atributo dependa de la clave, de toda la clave y de nada mas que la clave.

Paso 5. Diseno del modelo entidad-relacion

- Construye el diagrama ER con entidades, atributos, claves y cardinalidades.
- Incluye como minimo: clientes, vendedores, ventas, detalle_venta, productos y categorias.
- Puedes agregar metodos_pago o ciudades si tu justificacion lo requiere.
- No uses la tabla ventas_crudas como tabla transaccional final.

Paso 6. Implementacion fisica en PostgreSQL

- Crea el esquema normalizado con CREATE TABLE.

- Agrega PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK cuando corresponda.
- Inserta los datos normalizados derivados de la tabla cruda.
- Evita duplicar clientes, vendedores, categorias y productos.

Paso 7. Validacion con consultas SQL

- Reconstruye el total de cada venta desde el detalle.
- Consulta productos mas vendidos por cantidad.
- Consulta ventas por vendedor.
- Consulta historial de compras por cliente.
- Verifica que no existan registros huérfanos en detalle_venta.

7. Plantilla de analisis de dependencias funcionales

Completa esta tabla en tu informe antes de construir el modelo final:

Dependencia funcional	Tipo	Accion de normalizacion
cliente_doc -> cliente_nombre, cliente_email, cliente_telefono, cliente_direccion	Por completar	Por completar
vendedor_id -> vendedor_nombre, vendedor_zona	Por completar	Por completar
producto_codigo -> producto_nombre, categoria	Por completar	Por completar
venta_id -> fecha_venta, cliente_doc, vendedor_id, metodo_pago	Por completar	Por completar
venta_id + producto_codigo -> cantidad, precio_unitario, descuento	Por completar	Por completar

8. Entregables del aprendizaje

Entregable	Contenido minimo	Formato sugerido
Informe de normalizacion	Diagnostico, dependencias funcionales, pasos 1FN, 2FN y 3FN, justificacion de decisiones.	PDF o Markdown
Diagrama ER	Entidades, atributos, PK, FK y cardinalidades.	PNG/PDF exportado desde diagrams.net
Script 01 - modelo	DROP/CREATE SCHEMA, CREATE TABLE, constraints y comentarios.	01_modelo.sql
Script 02 - datos	INSERT de datos normalizados sin duplicidades innecesarias.	02_datos.sql
Script 03 - validacion	Consultas solicitadas y evidencias de resultados.	03_consultas.sql
README	Instrucciones para ejecutar los scripts y decisiones principales.	README.md

9. Consultas minimas de validacion

El modelo final debe permitir responder estas preguntas con SQL. No basta con dibujar el modelo.

- 1. Total calculado por cada venta desde el detalle.
- Resultado esperado: venta_id, total_calculado.

- 2. Productos mas vendidos por cantidad total.
- Resultado esperado: producto_codigo, producto_nombre, unidades_vendidas.

- 3. Ventas por vendedor.
- Resultado esperado: vendedor_id, vendedor_nombre, cantidad_ventas, valor_total.

- 4. Historial de compras de un cliente especifico.
- Resultado esperado: cliente_doc, cliente_nombre, venta_id, fecha_venta, producto, cantidad.

- 5. Control de integridad: detalles sin venta o sin producto.
- Resultado esperado: cero registros huérfanos.

10. Restricciones tecnicas obligatorias

- Usar PostgreSQL como motor de base de datos.
- Crear un esquema propio, por ejemplo normalizacion_ventas.
- No modificar manualmente los datos para esconder inconsistencias; las transformaciones deben justificarse.
- No dejar campos multivaluados en el modelo final.
- No guardar total_venta como unica fuente de verdad. Si decides almacenarlo, debes justificar la decision y validar su consistencia.
- Usar claves foraneas para proteger la integridad referencial.
- Usar CHECK para validar cantidad mayor que cero y precios no negativos.
- Usar UNIQUE donde existan identificadores naturales como documento de cliente o codigo de producto.

11. Rubrica de evaluacion

Criterio	Puntos	Indicadores de logro
Diagnostico de la tabla cruda	15	Reconoce redundancias, atributos no atomicos, anomalias y dependencias funcionales.
Aplicacion de 1FN	15	Elimina grupos repetitivos y garantiza atributos atomicos sin perder informacion.
Aplicacion de 2FN	15	Elimina dependencias parciales y separa correctamente entidades principales.
Aplicacion de 3FN	15	Elimina dependencias transitivas y justifica catalogos o tablas auxiliares.
Modelo ER	15	Incluye entidades, claves, relaciones y cardinalidades coherentes con las reglas de negocio.
Implementacion SQL	15	Usa DDL correcto, PK, FK, NOT NULL, UNIQUE, CHECK e inserciones normalizadas.

Consultas de validacion	10	Demuestra que el modelo permite reconstruir totales y responder preguntas de negocio.
-------------------------	----	---

12. Niveles de desempeno

Nivel	Rango	Descripcion
Superior	90-100	Modelo consistente, bien justificado, sin campos multivaluados, con constraints completos y consultas correctas.
Alto	80-89	Modelo funcional con pequenos ajustes pendientes en justificacion, constraints o consultas.
Basico	70-79	Normalizacion parcial; hay entidades correctas, pero persisten redundancias o dependencias no resueltas.
Bajo	0-69	El modelo final conserva la tabla cruda como base principal o no protege integridad referencial.

13. Estructura sugerida de entrega

```
mini-proyecto-normalizacion-ventas/
├─ README.md
├─ informe_normalizacion.pdf
├─ diagrama_er.png
├─ sql/
│   ├── 00_tabla_cruda.sql
│   ├── 01_modelo_normalizado.sql
│   ├── 02_datos_normalizados.sql
│   └─ 03_consultas_validacion.sql
```

14. Entorno opcional con Docker Compose

Si el curso se trabaja por contenedores, el aprendiz puede montar PostgreSQL y pgAdmin en una carpeta independiente del modulo. La carpeta sugerida es modulo-2-normalizacion/entorno-postgres/ para separar infraestructura, scripts y evidencias.

```
# Estructura sugerida
modulo-2-normalizacion/
├─ entorno-postgres/
│   └─ docker-compose.yml
├─ mini-proyecto-normalizacion-ventas/
│   └─ sql/

# Comando sugerido desde la carpeta entorno-postgres
# docker compose up -d
```

15. Pistas pedagogicas sin entregar la solucion

- No empieces creando tablas al azar: primero escribe las dependencias funcionales.
- Si un dato se repite muchas veces y describe a una entidad, probablemente debe vivir en su propia tabla.
- Si una tabla de detalle tiene una clave compuesta, revisa dependencias parciales para aplicar 2FN.
- Si un atributo depende de otro atributo que no es clave, revisa dependencia transitiva para aplicar 3FN.
- El precio de un producto en una venta historica no siempre debe actualizarse cuando cambia el precio actual del catalogo.
- Un buen modelo permite reconstruir la venta original usando JOIN, pero sin guardar listas separadas por comas.

16. Checklist final antes de entregar

- ☐ La tabla final no contiene productos_codigos, cantidades o precios separados por comas.
- ☐ Cada tabla tiene una clave primaria clara.
- ☐ Las relaciones tienen claves foraneas activas.
- ☐ Las cantidades no aceptan valores menores o iguales a cero.
- ☐ Los productos no se duplican innecesariamente.
- ☐ Los clientes no se duplican por cada compra.
- ☐ El total de la venta se puede calcular con una consulta SQL.
- ☐ El diagrama ER coincide con los scripts SQL entregados.
- ☐ El README explica como ejecutar los scripts.
- ☐ Las consultas de validacion se ejecutan sin errores.

17. Nota para el instructor

Este mini proyecto esta disenado como actividad evaluable. El documento no incluye el modelo final resuelto para que el aprendiz realice el proceso de analisis, normalizacion, disenio e implementacion. Durante la socializacion, el instructor puede validar diferentes soluciones siempre que cumplan las formas normales, las reglas de negocio y la integridad referencial.